DESIGN SYSTEMS & AUTOMATION ARCHITECTURE

Критический анализ пайплайна: Pixso + DeepSeek Harness

Статус: Архитектурная ревизия Стек: Pixso MCP, shadcn/ui, DSH Фокус: Zero-Hallucination Pipeline

> **Ключевой вердикт ревизии:** Главная угроза пайплайна — не подбор библиотек, а иллюзия бесшовной трансляции Figma-форматов в Pixso и неконтролируемый генеративный хаос при прямом доступе LLM к рисованию на холсте. Автоматизация заработает стабильно только при строгом разделении на декларативный AST-манифест и детерминированный раннер.

1. Архитектурный аудит скрытых уязвимостей

Попытка дать большой языковой модели возможность свободно создавать и трансформировать векторные ноды на холсте Pixso неизбежно приведет к деградации интерфейсов. Ниже приведены 4 критические точки отказа предложенной схемы.

Компонент	Технический риск и физика сбоя	Инженерное решение (Fix)
Импорт .fig в Pixso `Obra / Figma Kits`	Движок импорта Pixso сплющивает цепочки *Variable Aliases* в сырой HEX. Теряются сложные ограничения Auto-layout (Min/Max limits) и семантические слоты инстансов.	Не полагаться на импорт переменных из файла. Выгружать токены в чистый JSON/CSS и накатывать их на коллекцию программно через `write_variables`.
Генерация макетов `DeepSeek Harness`	Прямое исполнение координат через `apply_design` / `eval_script` порождает «галлюцинации верстки», отрыв инстансов и хаос с отступами.	**Strict AST Pattern:** LLM выдает только JSON-дерево по жесткой схеме экранов. Холста касается исключительно детерминированный локальный скрипт-сборщик.
Управление контекстом `ds-manifest.json`	Скармливание полного дампа дизайн-системы (сотни вариантов и внутренних атомов) переполняет контекст и вызывает путаницу в пропсах.	Двухуровневый манифест: верхнеуровневый индекс молекул/организмов плюс ленивая загрузка детальных схем (On-demand Schema Fetching).
Среда исполнения `Pixso Local MCP`	Отсутствие rollback при ошибках `eval_script` и падения соединения при частых одиночных вызовах API к десктопному клиенту.	Сборка во временный изолированный фрейм (`_sandbox_temp`). Пакетная отправка вызовов (Batch API) и перенос на холст только после аудита.

2. Декларативный контракт сборки (AST-подход)

Для исключения отсебятины DeepSeek лишается доступа к геометрии и свойствам стилей. Модель оперирует исключительно семантическими компонентами и предопределенными паттернами экранов:

Контракт для генерации экрана авторизации (JSON-Schema):

```
{
  "pattern": "AuthCard",
  "slots": {
    "header": { "title": "Вход в систему", "description": "Введите рабочие учетные данные"
},
    "fields": [
      { "component": "Input", "props": { "label": "Email", "type": "email", "required": true
} },
      { "component": "Input", "props": { "label": "Пароль", "type": "password", "required":
true } }
    ],
    "submit": { "component": "Button", "props": { "variant": "default", "label": "Войти" } }
  }
}
```

3. Прототипирование: разделение ответственности

- **Микро-интерактивность (Компонентный уровень):** Состояния Hover, Focus, Pressed, Disabled жестко зашиваются в интерактивные варианты (`Component Sets`) внутри Pixso. Модель их не настраивает.
- **Макро-навигация (Сценарный уровень):** Харнесс формирует только направленный граф переходов экранов (`Screen_A.Button[Submit] -> NavigateTo(Screen_B)`). Раннер компилирует связи в Reactions API.

4. Пошаговый план Smoke-тестирования (Go / No-Go)

1. **Проверка целостности алиасов:** Залить в Pixso тестовый фрейм с кнопкой shadcn. Через MCP вызвать `read_variables`. Если поле `valuesByMode[modeId].type` не равно `VARIABLE_ALIAS`, импорт .fig бракуется в пользу программного развертывания токенов (Путь С).
2. **Проверка мутации инстансов:** Выполнить команду `create_instance`, программно подменить текстовый слой и переключить вариант с `default` на `destructive`. Убедиться, что семантический цвет изменился.
3. **Проверка переключения темы:** Создать родительский фрейм, поместить внутрь инстансы, сменить режим темы на `Dark`. Проверить автоматический пересчет стилей без отрыва токенов.